## Objectives:

The objective of this project is to to build a robust search system that can answer user queries effectively by experimenting with the various blocks in the system, such as

-Design and implement a Retrieval-Augmented Generation (RAG) system for answering user queries from PDF documents.

-Enable semantic search by converting textual data into embeddings and retrieving relevant content based on meaning rather than keywords.

-Improve answer quality by integrating:

-Document chunking strategies

-Embedding models

-Re-ranking mechanisms

-Prompt-based generation

-Build an interactive system that allows users to query documents and receive context-aware answers.

-Experiment with different configurations (e.g., chunk sizes) to enhance retrieval performance and system effectiveness.

The system is a modular RAG pipeline that processes documents into embeddings, retrieves relevant context using vector search and re-ranking, and generates answers using a language model.

## Design

Here's a clear and structured design of your RAG-based code, suitable for your project report:

### ⬍ System Design Overview

The system follows a Retrieval-Augmented Generation (RAG) architecture, consisting of sequential stages that transform raw documents into meaningful answers.

### ⬍ 1. Input Layer

User provides a query:

"Eosinophil value"

System acts as an interactive CLI-based query interface

### 🔹 2. Document Ingestion Layer

Input documents (PDFs) are loaded

Text is extracted using:

PyPDF2

Output:

Raw textual content from documents

### 🔹 3. Preprocessing & Chunking Layer

Documents are split into smaller chunks using:

RecursiveCharacterTextSplitter

Chunking parameters:

Variable chunk sizes (e.g., 300, 500, 800)

Overlap to preserve context

Output:

List of text chunks

### 🔹 4. Embedding Layer

Each chunk is converted into a vector representation using:

SentenceTransformer

Output:

High-dimensional embeddings for each chunk

### 🔹 5. Vector Database Layer

Embeddings are stored in:

ChromaDB

Enables:

Fast similarity search

Persistent storage

Output:

Searchable vector index

### 🔹 6. Retrieval Layer

When a query is entered:

Query is embedded

Similar chunks are retrieved from database

Output:

Top-K relevant chunks (e.g., top 5–10)

### 🔹 7. Re-ranking Layer

Retrieved chunks are re-ranked using:

Cross-encoder model

Purpose:

Improve relevance ordering

Output:

Top-N refined chunks (e.g., top 3)

### 🔹 8. Context Construction Layer

Top documents are combined into a single context:

Context = Result 1 + Result 2 + Result 3

### 🔹 9. Generation Layer (LLM)

Uses:

transformers pipeline (e.g., GPT-style model)

Prompt includes:

User query

Retrieved context

Output:

Final natural language answer

### 10. Output Layer

Displays:

Retrieved documents

Final generated answer

## End-to-End Flow

User Query

↓

Document Retrieval (Vector DB)

↓

Re-ranking

↓

Context Building

↓

LLM Generation

↓

Final Answer

## ⬘ Key Design Features

Modular pipeline (each step independent)

Supports experimentation (chunk sizes)

Combines:

Semantic search (retrieval)

Deep learning (generation)

### ⬍ Limitation in Design

Re-ranking is not answer-aware

Only one embedding model used

No hybrid search (keyword + vector)

Prompt is static

It seems like I can't do more advanced data analysis right now. Please try again later.

## Technical Summary:

Stores documents, chunks, and embeddings in tabular form

Interacting with the file system

The project utilizes a combination of libraries to build a Retrieval-Augmented Generation (RAG) system:

Data Processing: pandas, os – for handling and organizing document data

Document Parsing: PyPDF2 – for extracting text from PDF files

Text Chunking: langchain_text_splitters – for splitting large text into manageable chunks

Embeddings & Semantic Search:sentence-transformers – for generating vector embeddings of text

Vector Database: ChromaDB – for storing and retrieving embeddings efficiently

Language Model (Generation): transformers – for generating answers using a pre-trained model

Supporting Dependencies: opentelemetry, rfc3987 – required for proper functioning of ChromaDB

Poppler (dependency for pdf2image) and Tesseract (OCR engine)

Python libraries for OCR

Steps:

Loads PDF files from a directory, extracts text using PyPDF2, and falls back to OCR if no text is found.

Returns a DataFrame.

Loads PDF files from a directory, extracts text, and returns a DataFrame.

## Implementation

- Load input documents (PDFs) using **PyPDF2**
- Extract and clean raw text from documents
- Split text into smaller chunks using **RecursiveCharacterTextSplitter**
    - Experiment with different chunk sizes (e.g., 300, 500, 800)
- Generate vector embeddings for each chunk using **SentenceTransformer**
- Store embeddings in a vector database using **ChromaDB** for efficient similarity search
- Accept user query through an interactive interface
- Convert user query into embedding using the same model
- Retrieve top-K relevant document chunks from the vector database
- Apply **re-ranking (CrossEncoder)** to improve relevance of retrieved results
- Select top-N documents (e.g., top 3) after re-ranking
- Construct context by combining retrieved document chunks
- Pass query + context to a **transformer-based language model**
- Generate final answer using the LLM
- Display:
    - Retrieved documents
    - Final generated answer

## Challenges Faced:

The implementation faced several challenges, including handling vague user queries that lacked sufficient context, which made accurate retrieval difficult. The re-ranking mechanism prioritized semantic similarity rather than exact answer correctness, often ranking less precise results higher than the actual answer-containing documents. Additionally, extracting text from PDFs introduced noise and formatting issues, affecting chunk quality and retrieval performance. Choosing an optimal chunk size required balancing context preservation and retrieval precision. The use of a single, general-purpose embedding model limited domain-specific understanding, particularly for medical data. The system also relied on a static prompt and lacked answer-aware retrieval mechanisms, reducing overall accuracy. Furthermore, experimentation was limited to chunk size without exploring other parameters, and the overall pipeline introduced latency due to repeated embedding and re-ranking processes.

**3 Models were changed to increase the accuracy level**

distilgpt2 -> gpt-2 -> gpt-4o-mini

Only text extractor did not help to resolve the embedding from the medical files**, but OCR was needed to retrieve the text** from images.

## Lessons Learned:

- Importance of **effective chunking strategy** for balancing context and retrieval accuracy
- **Semantic similarity ≠ correct answer**
    - Retrieval and re-ranking do not guarantee answer correctness
- Need for **better query understanding** to handle vague user inputs
- Value of **clean and well-structured data**
    - Poor PDF extraction significantly impacts performance
- Importance of **experimenting with multiple parameters**
    - Not just chunk size, but also embeddings, prompts, and top-K
- Benefit of using **domain-specific models** (e.g., medical embeddings)
- Need for **answer-aware retrieval or filtering mechanisms**
- Role of **prompt engineering** in improving final answer quality
- Understanding the **limitations of re-ranking models**
- Importance of **modular pipeline design** for easy experimentation and improvement

## CODE:

https://drive.google.com/file/d/1znjaX4yAQfDWXu8f4wCkW4r1oWJjEZKt/view?usp=sharing